Ders 5 Genişletilmiş Notu: Gözetimsiz Öğrenme: K-Means, Hiyerarşik Kümeleme ve Uzaklık Fonksiyonları

Ders 5 Genişletilmiş Notu: Gözetimsiz Öğrenme: K-Means, Hiyerarşik Kümeleme ve Uzaklık Fonksiyonları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Çerçevesi: Gözetimli ve Gözetimsiz Öğrenme
- 2. Web Verisinde Kümelenecek Nokta Nedir?
- 3. Kümeleme Nerede İşe Yarar?
- 4. Kümeleme Algoritmalarının Kategorileri ve Ortak Gereksinimleri
- 5. K-Means Algoritması
- 6. Uzaklık Fonksiyonlarına İlk Bakış
- 7. K-Means Büyük Veriye Nasıl Ölçeklenir?
- 8. K-Means'in Zayıf Noktaları
- 9. Kümeleri Dışarıdan Bakan Birine Anlatmak
- 10. Hangi Algoritmayı Kullanmalıyım?
- 11. Hiyerarşik Kümeleme ve Dendrogram
- 12. Kümeler Arası Uzaklık: Single, Complete, Average Link ve Centroid
- 13. Veri Ön İşleme Hattı
- 14. Uzaklık Fonksiyonlarına Dönüş: Sayısal, Binary ve Nominal Öznitelikler
- 15. Kümeleme Kalitesinin Değerlendirilmesi
- 16. Konuların Web Madenciliği Projesine Bağlanması
- 17. Ders Sonu Soru-Cevap

1. Dersin Çerçevesi: Gözetimli ve Gözetimsiz Öğrenme

Ders kitabı (Web Data Mining) bölüm bölüm işleniyor ve bu derste kitabın dördüncü bölümündeyiz. Hoca, dönem sonuna kadar kitabın web kullanım madenciliği (web usage mining) bölümüne kadar gitmeyi planlıyor. Hoca bunun sanıyorum 12. bölüm olduğunu söyledi. İlk üç bölüm önceki derslerde konuşuldu. Bu ders ise gözetimsiz öğrenmenin (unsupervised learning) hızlı bir tekrarıdır. Soru şudur: Gözetimsiz öğrenmeyi web verisi üzerinde nasıl uygularız, ne tip veriyle çalışırız, hangi yöntemleri kullanırız?

Önceki hafta gözetimli öğrenme (supervised learning) konuşulmuştu. Gözetimli öğrenmede etiketli veriden bir model öğrenilir. Sonra bu model, etiketi olmayan yeni bir veriyi etiketlemek için kullanılır. "Supervised" sözcüğünün karşılığı da budur: Modeli eğiten, etiketli veridir.

Gözetimsiz öğrenmede böyle bir durum yoktur. Veri etiketsizdir ve önceden bir model oluşturulmaz. Doğrudan veri üzerinde çalışılır ve verinin içindeki yapı anlaşılmaya çalışılır. Buradaki "yapı", verinin içinde kaç tane gruplanabilecek topluluk olduğudur. Bu konu veri madenciliği alanında çok çalışılmıştır. Web madenciliğinde aynı teknikler webden elde edilen veriye uygulanır.

2. Web Verisinde Kümelenecek Nokta Nedir?

2.1 Veri Türleri

Hoca önce web verisinin türlerini saydı:

- **Tıklama akışı (clickstream) verisi:** Sayfada gezinirken oluşturduğumuz gezinme (browsing) verisidir.
- **Oturum (session) verisi:** Sayfa üzerinde oluşturduğumuz oturum verisidir. Tıklama akışı verisi zaten oturum bazlı elde edilir. Böylece bir oturumda insanların ne tür hareketler yaptığı anlaşılmaya çalışılır.

Kümeleme algoritmaları bu oturum bazlı tıklama akışı verisi üzerinde çalıştırılır.

2.2 İki Boyuttan n Boyuta

En basit örnek iki boyutlu uzaydaki noktalardır. Slayttaki görsele bakınca noktaların üç grupta özetlenebileceği gözle görülür. Ama web madenciliği problemlerinde her nokta n boyutludur. Boyutların her biri de farklı tipte bir değişken olabilir: Kimisi tam sayı (integer), kimisi boolean değerdir. Yalnız boyut sayısı artmaz. Üzerinde çalışılması gereken veri sayısı da artar.

2.3 Web Oturumundan Davranış Vektörüne

Hoca web oturumunu şöyle anlattı. Bir internet sitesine girdiğiniz andan siteden çıktığınız, yani tarayıcıyı kapattığınız ana kadar bir oturum oluşturursunuz. Süreç şöyle işler:

- Sayfa, tarayıcıya bir çerez (cookie) bırakır.
- Çerezde kimliğiniz ve o anki oturum kimliğiniz (session ID) bulunur.
- Gönderdiğiniz her HTTP isteğinde (request) oturum kimliğiniz ve kullanıcı kimliğiniz karşı tarafa gider.
- Karşı taraf, oturum süresince gelen bütün HTTP isteklerini oturum kimliği üzerinden birbiriyle ilişkilendirir.

Aynı oturum kimliğiyle gelen tıklama verisinden, o oturumdaki davranışınızı belirten bir vektör oluşturulur. Bu vektörün elemanları, hareketlerinizden çıkarılmış istatistiksel değerlerdir. Örnekler:

- Oturumda kaç sayfa görüntüleme (page view) yapıldığı
- Kaç tıklama yapıldığı
- Kaç ürün görüntülendiği

Çok sayıda kullanıcı çok sayıda oturum oluşturur. Her oturum n boyutlu uzayda bir nokta olarak düşünülürse, bu noktalar kümelendiğinde şu soruya yanıt verilebilir: Sayfamız üzerindeki insanların davranışları gruplanabiliyor mu, benzer davranışlar var mı?

3. Kümeleme Nerede İşe Yarar?

Hoca üç örnek verdi.

Tişört üretimi. İnsanlara tişört üretmek istiyorsunuz. Herkese uygun ayrı bir tişört üretmek imkânsızdır. Bunun yerine insanlar gruplanır. Kaç grup varsa, her gruba uygun bir büyüklükte tişört üretilir.

Pazarlama. Pazarlama birimleri özellikle kullanıcı profilleme problemi üzerinde çok çalışır. Ulaşılması gereken bir milyon müşteri varsa, her birine ayrı kampanyayla gitmek imkânsızdır. Müşteriler gruplanır ve grup profilleri oluşturulur. Kampanya gruplara yönelik belirlenir ve o gruplardaki insanlara e-posta, telefon ya da SMS ile gönderilir.

Doküman kümeleme. Web madenciliğinde yalnız kullanıcı ile web sitesi arasındaki davranışı anlamaya çalışmayız. Web sitelerindeki dokümanları tarayıp (crawl) indirdikten sonra da bunların hiçbirinin etiketi yoktur. Ayrıştırıp (parse) içeriğini çıkarabiliriz, ama bu yetmez. Dokümanları gruplamak isteriz: Elimizdeki bin dokümanın kaçı spor, kaçı eğitim, kaçı bilim, kaçı politika alanındadır?

Bunun için her doküman bir vektöre dönüştürülür. Bir dildeki kelimeler bir vektör uzayı oluşturur. Bir dokümanda geçen kelimelerin karşılığına 1, geçmeyenlerin karşılığına 0 yazılır. Böylece her doküman, kelime sayısı kadar boyutu olan bir vektörle tanımlanır. Bu vektörler kümelenir. Oluşan gruplara bakılır ve hangi konuda oldukları içlerindeki kelimelerden anlaşılır.

> **Hocanın vurgusu:** Bu problemlerin hepsinde yapılan iş aynıdır: Etiketi olmayan, elemanları sayısal değerlerle ifade edilen n boyutlu vektörlerden oluşan noktaları kümelemek. Bundan sonra anlatılan her şey bu problem üzerinedir.

4. Kümeleme Algoritmalarının Kategorileri ve Ortak Gereksinimleri

4.1 İki Ana Kategori

Atama tabanlı (bölümleme, partitional) algoritmalar. Önce bölütler (partition) oluşturulur. Sonra gruplanacak kalan noktalar bu gruplara atanır. Yuvarlak ya da eliptik şekilli grupları bulmakta oldukça başarılıdır. Hesaplama maliyeti (computation cost) daha düşüktür, ama bir takım zayıf noktaları vardır.

Hiyerarşik (hierarchical) algoritmalar. Noktalar birbiriyle karşılaştırılıp birleştirilerek ilerlenir. Belirli bir şekle sahip olmayan grupları bulmakta faydalıdır. Her aşamada bütün noktalar birbiriyle karşılaştırıldığı için hesaplama maliyeti daha yüksektir. Avantajı farklı şekillerdeki grupları tespit edebilmesidir.

Bunların dışında dağılım (distribution) tabanlı ve yoğunluk (density) tabanlı algoritmalar da vardır. Hoca bu derste yalnız ilk iki kategoriden söz etti.

4.2 Uzaklık Fonksiyonu Zorunludur

Her iki kategoride de iki nokta arasındaki uzaklık hesaplanır ve başka uzaklıklarla karşılaştırılır. Bu yüzden bir uzaklık fonksiyonu mutlaka tanımlanmalıdır.

4.3 Kaliteyi Ölçmek Gerekir

Kümeler oluştuktan sonra sonuca "iyi" ya da "kötü" diyebilmek gerekir. Bunun için kümeleme kalitesini belirleyen metrikler vardır. Etiketleri bilmediğimiz için iki şeye bakarız:

- **Küme içi uzaklık (intra-cluster distance):** Bir kümedeki noktaların birbirine olan ortalama uzaklığıdır. En aza indirilmek istenir. Kümenin elemanları birbirine yakın olmalıdır.
- **Kümeler arası uzaklık (inter-cluster distance):** İki farklı küme arasındaki uzaklıktır. En büyük yapılmak istenir.

Mümkünse algoritma, birbirinden ayrık ve kendi grubundaki bütün noktaları yakalamış kümeler bulmalıdır.

> **Ders sonrası not:** Altyazıda hoca birinci tanımı da "intercluster distance" diye söylüyor (00:37:14). Tanım küme içi uzaklığa aittir. Doğrusu "intra-cluster distance"tır. Hoca büyük olasılıkla "intra" demiş, altyazı yanlış yazmış olabilir.

5. K-Means Algoritması

5.1 K Değeri

Atama tabanlı algoritmaların en yaygını K-Means'tir. Elimizde çok boyutlu vektörlerden oluşan bir veri seti vardır. Algoritmanın adından da anlaşıldığı gibi en başta bir k sayısı belirlenir: "Bu veri setinde k küme olduğunu düşünüyorum ve gruplamayı buna göre yapacağım."

Baştan böyle bir şey söylemek zordur. Bu yüzden en uygun k değerini bulan yöntemler vardır: Elbow ve Silhouette. Bunlar farklı k değerleri için, örneğin $k = 2$'den $k = 10$'a kadar, kümeleme yapar. Sonra oluşan kümelemenin kalitesine bakarak en uygun k değerini bulmaya çalışır. Anlatım için k değerinin elimizde olduğunu varsayalım.

5.2 Algoritmanın Akışı

K-Means tekrarlı (iterative) bir algoritmadır. Önceden belirlenmiş kriterler sağlanana kadar döngü sürer. Slayttaki iki merkezli örnek üzerinden akış şöyledir:

1. Veri setinden rastgele k nokta seçilir. Bu noktalara merkez (centroid) denir. Örnekte iki nokta seçilmiştir.
2. Kalan bütün noktalar merkezlere atanır.
3. Atanan noktalar kullanılarak merkezler yeniden hesaplanır.
4. Durma kriterleri sağlanıyor mu diye bakılır. Sağlanmıyorsa ikinci adıma dönülür ve bütün noktalar yeni merkezlere atanır.

Dikkat edilecek bir nokta vardır. İlk adımda merkezler gerçek veri noktalarıdır. Sonraki iterasyonlarda hesaplanan merkezler ise sanal merkezlerdir, yani gerçekte var olmayan noktalardır.

5.3 Sözde Kod

Hoca sözde kodu (pseudocode) ekranda gösterip sözlü olarak anlattı. Anlatımından kurulan yapı şöyledir:

```
Girdi: veri seti D, küme sayısı k
D içinden k veri noktasını başlangıç merkezi seç
tekrarla:
    D içindeki her x noktası için:
        x'in k merkezin her birine uzaklığını hesapla
        x'i en yakın merkeze ata
    her küme için yeni merkezi hesapla
durma koşulu sağlanana kadar
```

İçteki döngü veri setini baştan sona tarar. Birinci veri noktasının k merkeze uzaklığı hesaplanır ve nokta en yakın merkeze atanır. Sonra ikinci noktaya geçilir. İç döngü bitince atamalar tamamlanmıştır. Her küme için yeni merkez hesaplanır ve durma kontrolü yapılır.

5.4 Durma Koşulları

Birden fazla durma koşulu vardır:

- **Atama değişmiyor:** Yeni iterasyonun başındaki atama ile önceki iterasyonun başındaki atama arasında fark yoksa ya da fark çok azsa durulur.
- **Merkezler değişmiyor:** İki iterasyon sonunda hesaplanan merkezler arasında fark yoksa ya da fark çok azsa durulur.
- **Hata kareleri toplamı değişmiyor:** İki iterasyonun hata kareleri toplamı (sum of squared error, SSE) arasında değişiklik yoksa ya da çok azsa durulur.

SSE şöyle hesaplanır. Her küme için merkez ile o kümenin elemanları arasındaki uzaklıkların kareleri toplanır. Bu toplamlar bütün kümeler için toplanır ve o kümeleme için tek bir değer elde edilir:

$$SSE = \sum_{j=1}^{k} \sum_{x \in C_j} dist(x, m_j)^2$$

Burada C_j, j. kümeyi; m_j ise o kümenin merkezini gösterir. Aynı hesap her iterasyonun sonunda yapılır ve değerler karşılaştırılır.

6. Uzaklık Fonksiyonlarına İlk Bakış

Atama adımında bir noktanın hangi merkeze daha yakın olduğunu bulmak için uzaklık fonksiyonuna ihtiyaç vardır.

> **Hocanın vurgusu:** Uzaklık fonksiyonu bütün kümeleme analizi algoritmalarının olmazsa olmazıdır. Seçimi de önemlidir.

En bilinen fonksiyon Öklid (Euclidean) uzaklığıdır. Öklid uzaklığı aslında Minkowski uzaklığının özel bir hâlidir. Minkowski uzaklığında noktaların öznitelikleri arasındaki farkların h. kuvvetleri alınıp toplanır. En sonunda toplamın h. dereceden kökü alınır:

$$dist(x_i, x_j) = \left(\sum_{r=1}^{n} |x_{ir} - x_{jr}|^h \right)^{1/h}$$

- $h = 2$ ise Öklid uzaklığı elde edilir.
- $h = 1$ ise Manhattan uzaklığı elde edilir.

Bunlara ek olarak ağırlıklı Öklid uzaklığı (weighted Euclidean), kare Öklid uzaklığı (squared Euclidean) ve Chebyshev uzaklığı gibi metrikler de vardır. Chebyshev uzaklığında, iki noktanın bütün öznitelikleri arasındaki farklara bakılır. Birbirinden en uzak olan özniteliğin farkı, iki nokta arasındaki uzaklık olarak alınır:

$$dist(x_i, x_j) = \max_r |x_{ir} - x_{jr}|$$

En yaygın kullanılan Öklid uzaklığıdır. Ama bazı veri setlerinde başka metrikler daha başarılı sonuç verir. Hocaya göre veri analizinde yapılan iş de budur: Bir veri setinde tek bir algoritmanın başarılı olacağı söylenemez. Hem kümeleme algoritması hem de kullandığı uzaklık metriği parametre yapılır. Kümeleme sonuçlarının kalitesine sürekli bakılır. En yüksek kaliteyi veren algoritma ve metrik kullanılır.

7. K-Means Büyük Veriye Nasıl Ölçeklenir?

7.1 Hiyerarşik Algoritmalarla Karşıtlık

Atama tabanlı algoritmalar büyük veriye ölçeklenebilir. Hiyerarşik algoritmalara ise "bellek içi (in-memory) kümeleme algoritmaları" da denir. Çünkü hesaplama maliyetleri çok yüksektir ve genelde belleğe sığacak büyüklükteki veriye uygulanırlar. Başarıları yüksektir, eliptik ve yuvarlak dışındaki şekilleri de yakalarlar. Ama maliyet yüzünden veri büyüklüğü sınırlıdır.

7.2 Büyük Veri Sürümü

Büyük veri sürümünde de önce en uygun k bulunur ve k başlangıç noktası seçilir. Dışta bir döngü, içte veri setini tarayan bir döngü vardır. Her küme için ($j = 1, \ldots, k$) iki değişken tutulur:

- s_j: Kümeye atanan veri noktalarının öznitelik değerlerinin toplamı
- n_j: Kümedeki nokta sayısı

İç döngü şöyle çalışır. Birinci veri noktası alınır. $m_1, m_2, \ldots, m_k$ merkezleriyle karşılaştırılır ve en düşük uzaklıktaki küme bulunur. Nokta, o kümenin s_j toplamına eklenir ve n_j bir artırılır. Sonra bir sonraki noktaya geçilir. Bütün veri tarandıktan sonra her küme için merkez hesaplanır:

$$m_j = \frac{s_j}{n_j}$$

Sonra dış döngünün başına dönülür.

> **Ders sonrası not:** Hoca 00:58:33 civarında yeni noktanın "kümenin mevcut centroid değeriyle" toplandığını söyledi. Kendi tanımına göre (00:56:57) toplanan değer s_j'dir, merkez m_j değil. Merkez, tarama bitince s_j/n_j olarak hesaplanır.

Bu işlem için verinin belleğe getirilmesi gerekmez. Veri diskte dururken kümeleme yapılabilir. Atama tabanlı algoritmalar buna olanak verir. Hiyerarşik algoritmalarda ise algoritmanın verinin tümünü bellekte görmesi gerekir.

7.3 Karmaşıklık

K-Means'in hem uygulaması kolaydır hem de büyük veriye ölçeklenir. Karmaşıklık şöyle bulunur:

- Dış döngü, durma koşulu oluşana kadar sürer. Kaç iterasyon süreceği baştan bilinmez: 20, 30 ya da 50 olabilir. Bu sayıya t diyelim.
- Her veri noktası için k merkezle karşılaştırma yapılır.
- Bu karşılaştırma n veri noktası için tekrarlanır.

Sonuç olarak karmaşıklık $O(t \cdot k \cdot n)$ olur. k ve t küçük değerlerse algoritma veri sayısıyla doğrusal (linear) büyür. Bu yüzden diğer algoritmalara göre maliyeti düşüktür.

8. K-Means'in Zayıf Noktaları

8.1 Uzaklık Tanımlanabilmelidir

Algoritma sürekli uzaklık hesaplayarak ilerler. İki nokta arasındaki uzaklık uzaklık fonksiyonlarıyla bulunamıyorsa algoritma uygulanamaz.

8.2 K Baştan Verilmelidir

En uygun k baştan belirlenmelidir. Atama tabanlı algoritmalar çok yaygın olduğu için bu zayıflığı gidermek üzere optimum k algoritmaları önerilmiştir. Önce bunlarla k bulunur, sonra kümeleme yapılır.

8.3 Aykırı Değerlere Duyarlıdır

Aykırı değerler (outlier) kümeleme sonucunu olumsuz etkileyebilir. Slayttaki örnekte iki doğal grup ve uzakta tek bir nokta vardır. K-Means her aşamada merkezleri bulur ve noktaları merkezlere atayarak ilerler. Uzaktaki nokta yüzünden başta oluşan bir merkez o tarafta kalır. Sonuçta doğal olarak tek küme olması gereken noktalar tam ortadan kesilir. Oysa doğru sonuçta iki doğal küme bulunmalı, uzaktaki nokta aykırı değer sayılmalıdır.

Hoca iki çözüm verdi:

- **Aykırı değerleri çıkarmak:** K-Means'ten önce aykırı değer tespiti (outlier detection) yapılır ve bu noktalar veriden çıkarılır. Böylece daha kaliteli sonuç elde edilir.
- **Rastgele örnekleme (random sampling):** Bütün noktaları temsil edecek bir örnek rastgele seçilir. Aykırı değerin bu örneğe düşme olasılığı vardır, ama düşüktür. Çünkü aykırı değerler bütün veri içinde sayıca azdır. Zaten bu yüzden aykırı değer denir: Az sayıdadırlar ve bütün veriyi temsil etmezler. Kümeleme önce örnek üzerinde yapılır. Kalan noktalar sonra oluşan kümelere atanır.

8.4 Başlangıç Merkezlerine Duyarlıdır

Başlangıçta seçilen k noktanın (initial seeds) kendisi de başarıyı etkiler. Slayttaki örnekte gözle görülen iki küme vardır. Ama rastgele seçilen iki merkez birbirine çok yakın olduğu için algoritma bittiğinde yanlış iki küme bulunur. Merkezlerden biri bir gruptan, diğeri öbür gruptan seçilseydi istenen kümeler elde edilirdi.

Farklı seçim yöntemleri vardır. Hocanın anlattığı yöntem şöyledir:

1. İlk nokta seçilir.
2. Veri baştan aşağı taranır. Birinci noktaya en uzak nokta, ikinci merkez olarak alınır.
3. $k = 3$ ise bütün veri tekrar taranır. İlk iki noktaya en uzak olan nokta üçüncü merkez olarak alınır.

Yani başlangıç merkezlerini seçmek için de veri üzerinde bir tarama yapılır. Amaç nihai kümelemenin yüksek kaliteli çıkmasıdır.

> **Ek bilgi (derste anlatılmadı):** En uzak noktayı seçme yaklaşımı aykırı değerlere de açıktır. Çünkü aykırı değer çoğu zaman diğer noktalara en uzak olan noktadır ve

merkez olarak seçilebilir. Bu yüzden bu yöntem, aykırı değerlerin önceden ayıklandığı veride daha güvenlidir.

8.5 Eliptik Olmayan Şekilleri Bulamaz

K-Means eliptik ve yuvarlak şekilleri bulmada başarılıdır. Garip şekilleri yakalaması ise zordur. Bu tür şekiller için hiyerarşik kümeleme kullanılır. Hoca bu konunun örneğine aşağıda döndü.

9. Kümeleri Dışarıdan Bakan Birine Anlatmak

Kümeleme sonucunda oluşan kümeler özet veri olarak tanımlanabilmeli, dışarıdan bakan birine anlatılabilmelidir. Hoca dört yol anlattı.

9.1 Merkez ve Yarıçap

En yaygın yöntem, kümenin merkezini ve yarıçapını (radius) belirtmektir. Ayrıca küme içindeki noktalar için bir standart sapma (standard deviation) verilerek noktaların kümede nasıl dağıldığı ifade edilebilir.

9.2 Görselleştirme ve Hocanın Örnekleri

Görselleştirme, özellikle analitik uygulamalarında görülen bir yöntemdir. Hoca bu noktada kendi öğrencilerinin yakın zamanda yaptığı çalışmalardan örnekler gösterdi.

URL kümeleme. Kümelenen veri URL verisidir. Her URL önce parçalarına ayrılır, sonra sayısal değerler içeren bir vektöre dönüştürülür. Bu vektörler üzerinde kümeleme yapılır. Çıkan kümeler incelendiğinde, en büyük kümeden insanların sitede ağırlıklı olarak arama (search) işlevini kullandığı görülmüştür. Başka kümeler kampanyalara ve kategorilere bakan URL'lerden oluşur. Böylece insanların site üzerindeki davranışları görülebilir.

Gezinme yolları. İnsanlar bir sayfadan diğerine geçer. Bu geçişler birleştirilince bir grafik oluşur. Gezinme davranışı bir vektöre dönüştürülür ve vektör üzerinde sınıflandırma yapılır. Örneğin bir satırda kullanıcı önce C'ye, sonra D'ye, A'ya ve B'ye gitmiştir. Farklı sınıflandırma algoritmalarıyla insanların hangi sıralı eylem zincirlerini yaptığı tespit edilip gruplanabilir.

Hocanın bu örnekleri göstermekteki amacı şudur: Kümeleme algoritmaları web madenciliğinde birçok veri tipi üzerinde çalıştırılır. Dinamik URL'ler, gezinme davranışları ve oturum verileri bunlara örnektir.

9.3 Öznitelik Sınır Değerleri

Küme, özniteliklerin sınır değerleriyle kural gibi tanımlanabilir. Slayttaki örnekte üç küme ve iki boyut vardır:

- Birinci boyutu 2'den küçük olan elemanlar bir kümeyi oluşturur.
- Birinci boyutu 2'den büyük, ikinci boyutu 1,5'ten büyük olan elemanlar başka bir kümeyi oluşturur.

9.4 En Sık Tekrar Eden Elemanlar

Bir kümede en fazla tekrar eden elemanlar da kümeyi tanımlamak için kullanılabilir. Doküman örneğini hatırlayalım: Dokümanlar kelime vektörleriyle ifade edilip gruplanmıştı. Her grupta en fazla tekrar eden kelimelere bakılarak grubun konusu (topic) belirlenebilir.

10. Hangi Algoritmayı Kullanmalıyım?

K-Means'in pek çok türevi önerilmiştir. Ama atama tabanlı algoritmaların en büyük problemi eliptik ya da yuvarlak olmayan grupları yakalayamamalarıdır. Slayttaki örnekte gözle görülen iki grup vardır: İçeride bir grup ve onu kapsayan dış bir grup. İstenen, içerdekini ve dışarıdaki kapsayan şekli ayrı ayrı yakalamaktır. K-Means ise bunun yerine istenmeyen iki grup bulur. Böyle şekiller varsa başka tipte algoritmalar denenmelidir.

> **Ek bilgi (derste anlatılmadı):** K-Means her noktayı en yakın merkeze atadığı için iki küme arasındaki sınır, iki merkeze eşit uzaklıktaki noktalardan oluşur. Bu nedenle kümeler dışbükey (convex) bölgeler olur. İç içe halkalar gibi dışbükey olmayan şekiller bu yapıya sığmaz.

Hoca şu soruyu sordu: Elimde web oturum verisi var ve grupları tespit etmek istiyorum, hangi algoritmayı kullanmalıyım?

> **Hocanın vurgusu:** Farklı kategorilerden birer temsilci algoritma seçilmeli, bu algoritmalar farklı uzaklık fonksiyonlarıyla veri setine uygulanmalı, sonuçlar kümeleme kalite metrikleriyle (intra-cluster ve inter-cluster distance) incelenmelidir.

Bunun nedeni, hangi algoritmanın başarılı olacağının baştan bilinmemesidir. Veride eliptik şekiller mi, yoksa birbirinin içine girmiş grupların oluşturduğu yapılar mı olduğu baştan bilinmez.

11. Hiyerarşik Kümeleme ve Dendrogram

11.1 Dendrogram ve İki Yön

Hiyerarşik algoritmalar çalışırken bellekte ağaç yapısında bir veri yapısı oluşturur. Bu yapıya dendrogram denir. Algoritma, verideki küme sayısını bu yapıyı kullanarak çıkarır. Ağaçta iki yönde ilerlenebilir:

- **Birleştirici kümeleme (agglomerative, bottom-up):** En alttan başlayıp yukarı çıkılır.
- **Bölücü kümeleme (divisive, top-down):** En üstten başlayıp aşağı inilir.

Yaygın olan birleştirici kümeleme olduğu için hoca ondan bahsetti.

11.2 Algoritma

Algoritmanın güzel bir tarafı vardır: Girdi olarak yalnız veri seti verilir. Kaç küme olduğunu baştan söylemek gerekmez.

1. Her veri noktası bir küme sayılır.
2. Kümeler arasındaki bütün ikili uzaklıklar hesaplanır.
3. Birbirine en yakın iki küme birleştirilir (merge) ve bu bilgi dendrograma eklenir.
4. Eldeki kümeler için ikili uzaklıklar yeniden hesaplanır.
5. Tek küme kalana kadar üçüncü adıma dönülür.

11.3 Beş Noktalı Örnek

Elimizde P1, P2, P3, P4, P5 noktaları olsun. Başlangıçta her nokta bir kümedir. Uzaklıklar hesaplanır: P1 ile P2, P3, P4, P5 arasındaki uzaklıklar bulunur. Aynı şey P2 ve diğerleri için de yapılır. Birleştirmeler sırayla şöyledir:

İterasyon	Birleşenler	Yeni küme	İşlem sonunda kümeler
1	P1, P2	1 numaralı küme	{P1, P2}, {P3}, {P4}, {P5}
2	P4, P5	2 numaralı küme	{P1, P2}, {P3}, {P4, P5}
3	P3, 1 numaralı küme	3 numaralı küme	{P1, P2, P3}, {P4, P5}
4	3 ve 2 numaralı kümeler	Tek küme	{P1, P2, P3, P4, P5}

Birinci birleştirmeden sonra bir küme iki elemanlı olur. Bu kümeyi temsil eden orta noktayı bulmak gerekir. Hoca bu örnekte merkezi (centroid) kullandı: P3 ile 1 numaralı küme arasındaki uzaklık, P3 ile kümenin merkezi arasındaki uzaklıktır. P4 ve P5 için de aynısı yapılır.

11.4 Dendrogramın Kesiti

Her birleştirme sonunda elimizde bir kümeleme sonucu vardır. En iyi kümeleme için dendrogramın farklı yüksekliklerinden bir çizgi çekildiği düşünülür. Her iterasyondan sonra o andaki

yapının kalitesine (intra-cluster ve inter-cluster değerlerine) bakılır ve sayısal bir değer üretilir. Örneğin ikinci aşamada üç küme, üçüncü aşamada iki küme vardır. Kalitesi en yüksek olan kesit alınır ve sonuç olarak kullanılır.

12. Kümeler Arası Uzaklık: Single, Complete, Average Link ve Centroid

Birleştirme kararı iki küme arasındaki uzaklığa göre verilir. Bu uzaklığın neye göre belirleneceği önemlidir. Az önceki örnekteki merkez yöntemi bunlardan biridir. Diğer üç yöntem aşağıdadır.

12.1 Single Link

İki küme arasındaki uzaklık, kümelerin birbirine en yakın iki noktası arasındaki uzaklıktır. Slayttaki örnekte iki grup birleşmiş olsun ve aralarında birkaç nokta kalsın. Bu noktalardan birinin hangi kümeye atanacağına karar verilirken, her kümede o noktaya en yakın olan nokta bulunur. Karşılaştırma bu en yakın noktalarla yapılır.

Algoritmanın işleyişini görmek için belirlenmiş kümeleri unutup yalnız noktaları düşünelim. Bütün noktalar birer kümedir. İkili uzaklıklara bakılır ve en yakın ikili birleştirilir. Sonra yeni küme ve tekli noktalar arasında yine en yakın ikili bulunur. Uzaklık hesaplanırken ikili kümede dikkate alınan nokta, hesaplanan noktaya en yakın olandır. Böylece noktalar birer birer kümeye eklenir.

Zincir etkisi (chain effect). Slayttaki örnekte aslında iki temel küme vardır. Ama aralarında birbirine çok yakın noktalar bulunur. Single link yöntemi yalnız en yakın noktalara baktığı için algoritma bu zinciri takip eder: Şu nokta kümeye yakın diye eklenir, sonra onun yanındaki, sonra onun yanındaki eklenir. Sonuçta doğal iki küme bulunamaz. Single link yuvarlak olmayan şekillerde başarılı olabilir. Ama iki farklı şekil arasında böyle bir zincir varsa başarısız olur.

12.2 Complete Link

İki küme arasındaki uzaklık bulunurken kümelerin birbirine en uzak iki noktası (two furthest data points) kullanılır. Tekli bir nokta ile bir küme karşılaştırılıyorsa, kümede o noktaya en uzak olan nokta seçilir. Slayttaki örnekte algoritma önce en yakın ikilileri birleştirir ve iki taraf kendi içinde büyür. Aradaki bir nokta iki kümenin de en uzak noktasıyla karşılaştırılır ve hangisine daha yakınsa ona eklenir. Bu yöntem, doğal iki küme arasında birbirine yakın noktalardan oluşan bir zincir olsa bile başarılı sonuç verir.

Complete link yönteminin başarısız olduğu yer aykırı değerlerdir. Uzak noktalarda tekli kümeler varsa başarılı kümeler oluşmayabilir.

12.3 Average Link

İki küme arasındaki uzaklık, kümelerdeki noktalar arasındaki bütün uzaklıkların ortalamasıdır. Tek bir nokta bir kümeyle karşılaştırılıyorsa, o nokta ile kümenin bütün elemanları arasındaki uzaklıklar bulunup ortalaması alınır. Aynısı diğer küme için de yapılır. Nokta, ortalama uzaklığı en küçük olan kümeyle birleştirilir.

Soru: Average link'te kümenin orta noktasını bulup yalnız onunla karşılaştırsak olmaz mı?

Cevap: Bu, centroid yöntemidir ve ayrı bir uzaklık yöntemi olarak kullanılır. Centroid yönteminde tek tek noktalar arasındaki uzaklıklar yerine önce noktaların orta noktası bulunur ve karşılaştırma onunla yapılır. Average link'te ise uzaklıklar tek tek bulunur ve ortalaması alınır.

Average link'in hesaplaması daha fazladır. Ama bu yöntemle zincir etkisi ve aykırı değer etkisi en aza iner.

12.4 Seçim Üzerine

Veriden aykırı değerleri tespit edip atma imkânı varsa complete link hesaplama açısından tercih edilebilir, ama önce veri ön işlemenin iyi yapılmış olması gerekir.

Bu uzaklık yöntemlerinin hepsi kümeleme algoritmalarını uygulayan kütüphanelerde bulunur. Algoritma seçilebildiği gibi single, complete, average link ya da centroid de parametre olarak seçilebilir. Seçilen uzaklık fonksiyonu sonucu doğrudan etkiler. Veri ön işlemeden memnunsanız ve deneyiminiz varsa belirli bir yöntemi tercih edebilirsiniz. Emin değilseniz birden fazla uzaklık metriğiyle çalıştırıp sonuçlara bakmakta fayda vardır.

Hoca bunu az önce gösterdiği URL çalışmasıyla bağladı: Baştan hangi algoritmanın ve hangi uzaklık metriğinin başarılı olacağı bilinmiyordu. Bu yüzden hepsi denenip sonuçlar değerlendirildi.

> **Ders sonrası not:** Hoca hiyerarşik algoritmalara örnek olarak DBSCAN ve bisecting K-Means'i saydı (01:56:39). DBSCAN büyük olasılıkla hiyerarşik değil, yoğunluk tabanlı bir algoritmadır. Bisecting K-Means ise K-Means'i bölücü (top-down) biçimde kullanan bir türevdir. Hoca da 02:33:21 civarında iki algoritmayı karşılaştırırken bu ikisini ayrı kategorilere yerleştiriyor, ama altyazıda hangisinin hangi kategoride olduğu net değil.

13. Veri Ön İşleme Hattı

Soru: Algoritmaları çalıştırmadan önce veriye uygulanacak, aykırı değer temizleme gibi farklı yöntemler de var mı?

Cevap: Hoca soruyu özetleyerek yanıtladı. Önce dersin yapısını hatırlattı. Dersin ilk üç, en fazla dört haftası tekrar niteliğindedir. Kalan on haftada link analizi, içerik analizi, yapı analizi, kullanım madenciliği (usage mining) ve fikir madenciliği (opinion mining) üzerinde yoğunlaşılacaktır. Veri ön işleme altyapısının veri madenciliği ve makine öğrenmesi derslerinden geldiği varsayılır.

13.1 Veriyi Vektöre Dönüştürmek

Veri ister oturum verisi, ister dinamik URL'lerden oluşan veri, ister sıralı eylemlerden oluşan yol (path) verisi olsun, hepsi elemanları sayısal değerler olan vektörlere dönüştürülmeye çalışılır. Bunun için gömme (embedding) yöntemleri kullanılır: Word2Vec, Graph2Vec, Node2Vec. Bu yöntemler bir URL'yi, bir grafı, yani web verisindeki birim veriyi alıp vektörleştirir.

Vektörler alt alta konunca kümelenecek veri ortaya çıkar. Bazen vektörlerin etiketi de olur. Örneğin vektör bir yolu ifade ediyorsa ve bu yolun hangi davranışa karşılık geldiği biliniyorsa etiket vardır.

13.2 Hattın Adımları

Gözetimli de gözetimsiz de olsa veri ön işleme gerekir. Adımlar bir hat (pipeline) oluşturur:

- **Eksik veri tamamlama (missing data imputation):** Kolon bazında eksik veri olup olmadığına bakılır. Eksik değer, o kolondaki diğer değerlerden yola çıkılarak tamamlanır.
- **Aykırı değer tespiti:** Kutu grafiği (box plot) analizi gibi yöntemler kullanılır. Bulunan aykırı değerler, gözetimli öğrenmede eğitim verisinden, gözetimsiz öğrenmede kümelenecek veriden çıkarılır.
- **Normalizasyon:** Bir özniteliğin değerleri 0-1000, diğerinin 0-1 aralığındaysa bu istenen bir durum değildir. Öznitelik aralıkları normalize edilir.

Gözetimli öğrenmede şu adımlar da eklenir:

- **Öznitelik önemi:** Hangi özniteliğin daha önemli olduğu bilgi kazancı oranı (information gain ratio) gibi metriklerle hesaplanır.
- **Bağımlı kolonlar:** Ki-kare (chi-square) testi gibi yöntemlerle birbirine bağımlı kolonlar belirlenir ve çıkarılır.

Boyut küçültme de bazen gerekir. Çünkü başta belirlenen n özniteliğin hepsi öğrenme algoritması için gerekli olmayabilir. Bunun için temel bileşenler analizi (PCA) ya da faktör analizi kullanılır.

13.3 Hiperparametre Optimizasyonu

Bu algoritmaların her biri parametriktir. PCA, faktör analizi, ki-kare testi ya da bilgi kazancı farklı parametrelerle çalıştırılabilir. Parametreleri belirlemek için hiperparametre optimizasyonu (hyperparameter optimization) algoritmaları kullanılır. Böylece en başarılı kümeleme analizini ya da en başarılı tahmini yapacak parametreler seçilir.

Hoca bir öğrencisinin çalışmasını gösterdi. Bu çalışmada eksik veri tamamlama ve aykırı değer tespitinin tamamlandığı varsayılır. Hattın adımları şunlardır:

- Bilgi kazancı ve ki-kare testiyle önemli öznitelikleri bulan bir katman
- Faktör analizi ve PCA ile boyut küçültme
- En sonda sınıflandırma algoritması

Bütün hat, hiperparametre optimizasyonuyla sürekli çalıştırılır. Her iterasyonda algoritmaların aldığı metrikler değiştirilir.

> **Ders sonrası not:** 02:07:41 civarında altyazıda özniteliklerin boyutunun "büyütülüp" sınıflandırma yapıldığı geçiyor. Söz konusu olan boyut küçültme algoritmaları olduğu için doğrusu büyük olasılıkla "küçültülüp"tür.

Hocanın bunu göstermekteki amacı şudur: Temel algoritmalar gerçek veri üzerinde kullanılırken farklı algoritmaları farklı metrikler ve farklı parametre setleriyle çalıştırmak zorunludur. Veri bilimcilerin zamanını alan işler de bunlardır. Bu ders kapsamında veri, web madenciliği verisi olacak ve bu aşamalar adım adım uygulanacaktır.

14. Uzaklık Fonksiyonlarına Dönüş: Sayısal, Binary ve Nominal Öznitelikler

14.1 Sayısal Metriklerin Anlamı

Hoca, Minkowski uzaklığından çıkan Öklid ve Manhattan uzaklıklarını hatırlatıp kare Öklid uzaklığını ekledi. Kare Öklid uzaklığında iki noktanın öznitelikleri arasındaki farkların kareleri toplanır, kök alınmaz:

$$dist(x_i, x_j) = \sum_{r=1}^{n} (x_{ir} - x_{jr})^2$$

Ağırlıklı, normal ve kare Öklid uzaklığının her birinin farklı bir anlamı vardır. Birbirinden uzak olan noktalar için uzaklığın daha büyük çıkmasını istiyorsanız kare Öklid uzaklığını kullanabilirsiniz. Uzak iki noktanın uzaklığının sayısal değerine daha fazla ağırlık verilmiş olur, yani değer daha da büyür.

Chebyshev uzaklığı da benzer bir düşünceyle seçilir. Bir veri setinde iki noktanın farkını, öznitelikler arasında en yüksek farkı gösteren öznitelikle tanımlamak istiyorsanız bu metriği seçersiniz.

> **Hocanın vurgusu:** Uzaklık metriğini gerekçesini (justification) göstererek seçebiliyorsanız, kümeleme analizinin sonucu da daha başarılı olur.
>
> **Ders sonrası not:** Hoca 02:11:41'de kare Öklid uzaklığının birbirine yakın noktalara daha fazla ağırlık verdiğini söyledi, hemen ardından (02:11:48) uzak noktalar için söyleyerek düzeltti. Doğrusu ikincisidir.

14.2 Binary Öznitelikler ve Karışıklık Matrisi

Öznitelik değerleri sayısal olabileceği gibi binary de olabilir. Binary değerlerde uzaklık bulmak için karışıklık matrisi (confusion matrix) kullanılır. Bu matris sınıflandırmada doğruluk, F-skoru, kesinlik (precision) ve duyarlılık (recall) metriklerini belirlerken de kullanılır.

İki veri noktası x_i ve x_j olsun. Her hücre, ilgili kombinasyona sahip öznitelik sayısını gösterir:

```
              x_j = 1   x_j = 0
x_i = 1          a         b
x_i = 0          c         d
```

Yani a ikisinde de 1 olan, b yalnız x_i'de 1 olan, c yalnız x_j'de 1 olan, d ikisinde de 0 olan özniteliklerin sayısıdır. Uyuşmayanlar b ve c, uyuşanlar a ve d'dir.

14.3 Simple Matching (Simetrik Binary)

Simple matching uzaklığında pay, uyuşmayan (mismatch) öznitelik sayısı; payda, toplam öznitelik sayısıdır:

$$dist(x_i, x_j) = \frac{b + c}{a + b + c + d}$$

Derste verilen örnek:

- **Verilen:** İki nokta, toplam 7 öznitelik. Uyuşmayan öznitelik sayısı $2 + 1 = 3$'tür.
- **İşlem:** Pay 3, payda 7'dir.
- **Sonuç:** $dist = 3/7$.

  Hocanın vurgusu: Karışıklık matrisini ezberlemeye gerek yoktur. "Uyuşmayan öznitelik sayısı bölü toplam öznitelik sayısı" diye aklınızda kalsın.

14.4 Jaccard (Asimetrik Binary)

Bazı problemlerde değerler 1 ve 0 olsa da değerlerden biri diğerinden daha önemlidir. Örneğin 1, 0'dan daha önemli olabilir. Bu asimetrik binary özniteliklerde Jaccard katsayısı kullanılır. Hoca bunun web madenciliğinde yoğun kullanılan metriklerden biri olduğunu söyledi. Pay yine uyuşmayan öznitelik sayısıdır. Paydaya ise 0-0 olanların dışındaki bütün öznitelikler girer:

$$dist(x_i, x_j) = \frac{b + c}{a + b + c}$$

Önemi düşük olan değerin iki noktada da aynı olduğu durum göz ardı edilir.

Aynı örnekle hocanın hesabı:

- **Verilen:** Uyuşmayan öznitelik sayısı 3, toplam öznitelik sayısı 7.
- **İşlem:** Hoca 0-0 olanları çıkardığında paydanın 6 olduğunu söyledi.
- **Sonuç:** $dist = 3/6$.

  Ders sonrası not: Payda 6 ise örnekte tam olarak bir tane 0-0 özniteliği olmalıdır. Slayt altyazıda okunmadığı için bu doğrulanamadı. Örnek ders kitabından alınmışsa büyük olasılıkla iki tane 0-0 vardır. Bu durumda doğru sonuç $3/6$ değil, $3/5$ olur.

  Ders sonrası not: Hoca buradaki oranlara "coefficient" dedi. Formüller uzaklığı veriyor. Literatürde büyük olasılıkla "katsayı" sözcüğü benzerlik için kullanılır: Simple matching benzerliği $(a + d)/(a + b + c + d)$, Jaccard benzerliği $a/(a + b + c)$'dir. Uzaklık \$1 - \$ benzerlik olarak bulunur ve yukarıdaki formüllere eşittir. Yine "confusion matrix" yerine bu kullanımda büyük olasılıkla "contingency table" adı daha yaygındır.

14.5 Nominal Öznitelikler

Bir özniteliğin ikiden fazla değer alabildiği durumda nominal öznitelikten söz edilir, örneğin üç değer alabilir. Hoca öznitelik türlerini üçe ayırdı: sayısal (numeric), binary ve nominal. Nominal özniteliklerde de simple matching yaklaşımı kullanılır. Birbiriyle eşleşen özniteliklerin sayısına q, toplam öznitelik sayısına r diyelim. Toplamdan q çıkarılınca pay kısmındaki uyuşmayan sayısı elde edilir:

$$dist(x_i, x_j) = \frac{r - q}{r}$$

14.6 Kosinüs Benzerliği

İçerik madenciliğinde (content mining) çok kullanılacak bir fonksiyon da kosinüs benzerliğidir (cosine similarity). İki doküman arasındaki uzaklığa bakarken dokümanlar birer vektör olarak tanımlanır. Uzaklık, vektörler arasındaki açıdan yola çıkılarak hesaplanır. Açı küçükse dokümanlar birbirine daha benzer, açı büyüdükçe birbirinden daha uzaktır.

> **Ek bilgi (derste anlatılmadı):** Kosinüs benzerliği $\cos\theta = \dfrac{x_i \cdot x_j}{\|x_i\|\,\|x_j\|}$ ile hesaplanır. Pay vektörlerin iç çarpımı, payda boylarının çarpımıdır. Değer büyüdükçe benzerlik artar. Uzaklık olarak kullanılacağı zaman çoğunlukla $1 - \cos\theta$ alınır.

15. Kümeleme Kalitesinin Değerlendirilmesi

Hoca sözü yine aynı noktaya getirdi: Tek bir kümeleme algoritması bütün veri setlerinde başarılı değildir. Farklı kategorilerden algoritmalar seçilip farklı uzaklık fonksiyonlarıyla çalıştırılmalı, sonuçların kalitesine bakılmalıdır. Kaliteye iki türlü bakılır:

- **Etiketli veri varsa:** Entropi (entropy), saflık (purity), NMI, doğruluk gibi metrikler kullanılır.
- **Etiketli veri yoksa:** Küme içi ve kümeler arası metriklere bakılabilir. Etiket yoksa değerlendirme zorlu bir problemdir.

15.1 Entropi ve Saflık

Entropi belirsizliği ölçen bir metriktir. Bilgi kazancı konuşulurken de geçmişti. Belirsizlik varsa entropi yüksektir, yoksa düşüktür. Hoca bunu iki örnekle anlattı:

- **Telefon:** Telefonunuz çalıyor ve arayanı biliyorsunuz. Belirsizlik yoktur, entropi 0'dır. Bilinmeyen bir numara arıyorsa kim olduğunu bilmezsiniz. Belirsizlik vardır, hoca bu durumda entropinin 1 olduğunu söyledi.
- **Çuvaldaki bilyeler:** Çuvala elinizi atıp bilye alıyorsunuz. Bilyelerin hepsi aynı renkse entropi 0'dır. Yarısı beyaz, yarısı kırmızıysa elinize ne geleceğini bilemezsiniz ve entropi 1'dir.

Saflık ise kümedeki bütün noktaların aynı tipte olması durumunda yüksektir. Bu, kümeye aynı tipteki verilerin sokulabildiği anlamına gelir.

> **Hocanın vurgusu:** Formülleri ezberlemek yerine anlamlarını bu şekilde akılda tutmak daha uygundur.

> **Ek bilgi (derste anlatılmadı):** p_i, kümedeki noktaların i. sınıfa ait oranı olsun. Entropi $-\sum_i p_i \log_2 p_i$, saflık ise $\max_i p_i$ olarak hesaplanır. Küme tek sınıftan oluşuyorsa entropi 0, saflık 1'dir.

15.2 900 Dokümanlık Örnek

Bazen etiketli veri üzerinde de kümeleme yapılır. Örnekte etiketli 900 doküman vardır ve algoritma 3 küme oluşturmuştur.

Birinci küme. Hoca kümeye düşen noktaları "250, 270, 280" diye sayarak toplamın 280 olduğunu söyledi. Entropisi düşüktür, çünkü belirsizlik azdır. Küme ağırlıklı olarak tek tipte veriden oluşur. Saflığı da diğerlerine göre yüksektir. Bu kümenin kalitesi yüksektir.

Üçüncü küme. Hoca noktaları "210, 310, 340" diye sayarak toplamın 340 olduğunu söyledi. Entropi değeri en yüksek olan kümedir. Birinci kümede çok yüksek bir yüzdeyle tek tip veri varken, burada verinin yaklaşık üçte ikisi bir sınıfta, üçte biri başka sınıftadır. Yani saflığı birinci kümeden düşük, entropisi birinci kümeden yüksektir.

> **Ders sonrası not:** Hocanın sayımı büyük olasılıkla birikimlidir: Birinci küme $250 + 20 + 10 = 280$, üçüncü küme $210 + 100 + 30 = 340$. Bu okuma toplamla da tutarlıdır, çünkü ikinci kümeye $900 - 280 - 340 = 280$ doküman kalır. Bu dağılımlarla hesap (python ile doğrulandı) şöyle çıkar: Birinci kümede saflık $250/280 \approx 0{,}893$, entropi

$\approx 0{,}590$; üçüncü kümede saflık $210/340 \approx 0{,}618$, entropi $\approx 1{,}258$. Bu sonuçlar hocanın karşılaştırmasıyla aynı yöndedir.

Ders sonrası not: Hocanın "entropi 1" değeri iki eşit olasılıklı durum için doğrudur. Sınıf sayısı artınca üst sınır da artar, üç sınıfta $\log_2 3 \approx 1{,}585$'tir. Üçüncü kümenin entropisinin 1'i geçmesi bundandır. Telefon örneğinde ise arayan için bir olasılık dağılımı verilmediğinden "1" değeri yalnız sezgi amaçlıdır.

Kümeleme algoritması tek bir küme bulmaz. Algoritmanın başarısını ölçmek için bütün kümelerin entropi ve saflık değerlerinin ortalaması hesaplanır. Sonra diğer algoritmalar aynı veri setinde denenip aynı değerlere bakılır. Hedef entropiyi düşürmek, saflığı artırmaktır. Bu başarılırsa kümelere o kümede olması gereken baskın tipteki veriler konmuş demektir.

Ders sonrası not: Hoca yalnız "ortalama" dedi. Bütün kümeleme için büyük olasılıkla küme büyüklükleriyle ağırlıklı ortalama alınır.

15.3 Etiket Yoksa

Elde etiketli veri yoksa iki metriğe bakılır:

- **Küme içi yakınlık (intra-cluster cohesion, compactness):** Bir kümedeki noktaların birbirine ne kadar yakın olduğunu gösterir. Düşük olması istenir.
- **Kümeler arası ayrıklık (inter-cluster separation, isolation):** Kümelerin birbirinden ne kadar ayrık olduğunu gösterir. Büyük olması, kümelerin birbirinden iyi ayrıldığını gösterir.

  Ders sonrası not: Altyazıda ilk metrik "inter-cluster cohesion" diye geçiyor (02:29:02). Tanım küme içine ait olduğu için doğrusu "intra-cluster cohesion"dır.

Kümeleri görselleştirme şansı 2 ve 3 boyutlu uzayda vardır. n boyutlu uzayda bu zordur, bu yüzden başarıya çoğu durumda görsel bir grafikten bakılamaz. Yalnız 2-3 öznitelikli uzaylarda görselleştirme de değerlendirmede kullanılabilir.

16. Konuların Web Madenciliği Projesine Bağlanması

Soru: Bu anlatılanlar bir projeyle nasıl eşleştirilebilir?

Cevap: Hoca, gösterdiği çalışmalar üzerinden şu eşleştirmeyi yaptı. Kullanılan veri web verisi, yapılan analiz ise derste işlenen kümeleme yöntemleridir.

Dinamik URL çalışması. Adımlar şöyledir:

1. N11 sitesinden dinamik URL'ler yazılan bir kodla elde edildi.
2. Düzenli ifadeler (regular expression) kullanılarak URL tipleri belirlendi. Böylece elde etiketli bir veri oluştu.
3. URL'ler parçalarına ayrıldı ve Word2Vec ile vektöre dönüştürüldü.
4. K-Means ve bisecting K-Means gibi algoritmalarla kümeleme analizi yapıldı.

Burada derste işlenenlerin dışında eklenen şey, URL'yi vektöre dönüştüren Word2Vec kütüphanesidir.

Log dosyası çalışması. Log dosyasındaki URL'lerden sıralı örüntü madenciliği (sequential pattern mining) ile sık örüntüler (frequent pattern) bulundu. Kitabın birliktelik kuralları bölümünde GSP algoritması görülmüştü. Bu çalışmada onun "kardeş" algoritması PrefixSpan kullanıldı. Örüntüler Node2Vec ile vektöre dönüştürüldü ve kümeleme algoritmaları uygulandı.

Hoca bu çalışmaların akademik yayına dönüştüğünü söyledi. Web verisi kod depolarından ya da Stanford'un veri seti paylaşım ortamlarından elde edilebilir. Tıklama verisi ve log dosyaları buna örnektir. Bir sonraki derste anlatılacak olan, web verisini indiren tarama (crawler) algoritmaları da çalışılabilir: Derinlik öncelikli arama (depth-first search) ve genişlik öncelikli arama (breadth-first search) gibi. Temel beklenti, işlenen algoritmaların web verisi üzerinde çalıştırılmasıdır.

Burada kastedilen web verisi, kullanıcının web ortamındaki etkileşimi sonucu oluşan veridir: IIS sunucusu altında toplanan veri ya da bir uygulamanın ürettiği kullanıcı verisi gibi.

17. Ders Sonu Soru-Cevap

Soru: Crawler ile webden toplanan veri geçerli bir veri seti olur mu?

Cevap: Amaca bağlıdır.

- **Çalışmanın konusu crawler ise:** Değerlendirilen şey toplanan veri değil, crawler'dır. Crawler bir arama yapmak için kullanılır. Veriyi e-ticaret sitelerinden yerele indirip önüne hibrit bir arama motoru ya da en azından listeleme yapan bir kullanıcı arayüzü koyabilirsiniz. Gerçek zamanlı çalışmak isterseniz kullanıcı bir anahtar kelime girer. Sistem geri planda sayfalara gidip sonuçları alır, birleştirir ve hibrit biçimde gösterir.
- **Crawling yalnız veri indirmek içinse:** Esas iş, indirilen veri üzerindeki metin madenciliğidir (text mining). Örneğin Wikipedia'dan taranan sayfalar kümelenebilir.

Hoca veri indirmenin problemli olduğunu ekledi: Ne kadar veri indirilecek, indirilen veri sağlıklı mı? Bu nedenle webde hazır bulunan oturum verileri, URL verileri, IIS ya da Tomcat sunucularında toplanmış log verileri üzerinde çalışılabilir. İçerik madenciliği çalışmak isteyenler için de bazı veri setlerinde haber sitelerinden indirilmiş HTML sayfaları ya da bunlardan ayrıştırılmış metin dokümanları vardır.

Soru: E-ticaret sitelerindeki ürün yorumlarında bir markaya yönelik olumlu ya da olumsuz görüş analizi yapmak web madenciliğine girer mi? Metin madenciliği gibi görünüyor.

Cevap: Girer. Kitabın fikir madenciliği bölümü bu alandadır. Bir ürünün altındaki yorumlar analiz edilip insanların olumlu ya da olumsuz görüş bildirdiği çıkarılabilir. Ayrıca görünüş odaklı fikir madenciliği (aspect-oriented opinion mining) ile ürünün hangi alt özelliklerine önem verildiği yakalanabilir. Telefon örneğinde alt özellikler ses kalitesi, görüntü kalitesi, ekran büyüklüğü ve çözünürlüktür. Yorumlardaki bu özelliklerin önündeki olumlu ya da olumsuz sıfatlara dayanarak insanların o özellik hakkında ne düşündüğü bulunur. Kitapta bu algoritmayla ilgili güzel bir örnek vardır.

Soru: Film öneri sistemi yapılabilir mi? Bir veri setine demografik, TF-IDF ve Apriori yöntemlerini uygulayıp karşılaştırmak uygun olur mu?

Cevap: Öneri sistemi uygundur. Bunun için içerik tabanlı (content-based) ve işbirlikçi filtreleme (collaborative filtering) yöntemlerine bakılmalıdır. Bu yöntemler kullanıcı-ürün ilişkisine dayalı matrisler oluşturur. İnsanların ürünlere verdiği puanlara dayanarak, henüz puan vermedikleri ürünler için değer üretir: "Bu kişi bu filmi izleseydi şu puanı verirdi." Öneriler buna dayanır.

Apriori, TF-IDF ve işbirlikçi filtreleme ayrı şeylerdir. Hoca bu bileşimi fazla karmaşık buldu ve sadeleştirilmesini önerdi. Çerçevesi belli olmalı: Hangi veri kullanılacak, o veri üzerinde ne yapılacak? Bir veri setinde ürün ve kullanıcı bilgileri ile kullanıcıların verdiği puanlar varsa, bu puanlardan yola çıkarak içerik tabanlı ya da işbirlikçi filtreleme çalışılabilir.

İşbirlikçi filtreleme soğuk başlangıç (cold start) denen bir problemle karşılaşır. İçerik tabanlı filtreleme bu probleme belli ölçüde yanıt verebilir. Bu iki algoritmanın yanı sıra soğuk başlangıcı çözen bir yöntem de önerilebiliyorsa o da yapılabilir. Apriori ve TF-IDF eklemek ise önerilmedi.

> **Hocanın vurgusu:** İş ne kadar karmaşıklaşırsa sonradan pişman olma ihtimali o kadar artar. Ne kadar basitleştirilip yapılabilir ölçüye getirilirse çalışma o kadar rahat ilerler ve tamamlanabilir.

Dersin Toparlaması

Bu derste gözetimsiz öğrenmenin web verisine nasıl uygulandığı görüldü. Oturum, URL ya da doküman fark etmeksizin web verisi önce sayısal, n boyutlu vektörlere dönüştürülür. Sonra bu noktalar kümelenir. Atama tabanlı K-Means ucuzdur ve veri diskteyken bile çalışır, ama k değerine, aykırı değerlere, başlangıç merkezlerine ve eliptik olmayan şekillere duyarlıdır. Hiyerarşik kümeleme k istemez ve farklı şekilleri bulabilir, ama pahalıdır ve verinin bellekte olmasını gerektirir. Sonucu kümeler arası uzaklık yöntemi (single, complete, average link, centroid) ile uzaklık fonksiyonu (Minkowski ailesi, Chebyshev, simple matching, Jaccard, kosinüs) doğrudan etkiler.

Hocanın dersin başından sonuna tekrarladığı mesaj şudur: Tek bir doğru algoritma yoktur. Farklı kategorilerden algoritmalar farklı uzaklık fonksiyonlarıyla denenmeli, sonuçlar etiket varsa entropi ve saflıkla, yoksa küme içi yakınlık ve kümeler arası ayrıklıkla karşılaştırılmalıdır. Gelecek hafta web içerik madenciliğine (web content mining), bilgi erişimi (information retrieval) bölümüyle başlanacak, tarayıcılardan (crawler) da bahsedilecektir. Kosinüs benzerliği orada çok kullanılacaktır.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.